

Instituto Tecnológico de Costa Rica

Carrera de computación, San Carlos

Compiladores e Intérpretes

Grupo 51

Compilador Mini GO

Profesor:
Óscar Víquez Acuña

Estudiante:
Magdaleno Gómez Díaz — 2023395761

I Semestre 2026

Tabla de contenidos

1. Análisis del Lenguaje 3
1.1 Descripción general 3
1.2 Gramática 3
1.3 Tipos de datos 3
1.4 Código que se puede y que no se puede escribir 3
1.5 Comparación con Go 4
2. Soluciones e Implementación 4
2.1 Arquitectura general 4
2.2 Scanner y Parser (ANTLR4) 5
2.3 Tabla de símbolos 5
2.4 TypeChecker (chequeo de alcances y tipos) 6
2.5 Encoder — generación de código LLVM IR y objeto x86-64 6
2.6 Editor gráfico (Qt6) 7
2.7 Manejo de errores 8
3. Resultados Obtenidos 8
3.1 Componentes implementados 8
3.2 Funcionalidades no implementadas o limitadas 8
4. Conclusiones 9
5. Bibliografía 10

1. Análisis del Lenguaje

1.1 Descripción general

Mini GO es un subconjunto del lenguaje Go (versión 1.22) [2] con fines educativos. Conserva la sintaxis y la semántica básica de Go pero excluye sus características avanzadas. Todo programa parte de la función func main() dentro del package main y los punto y coma son explícitos y obligatorios.

1.2 Gramática

La gramática se especifica en ANTLR4 en el archivo grammar/MiniGo.g4. La regla inicial root reconoce “package” IDENTIFIER “;” seguido de una lista de declaraciones de variables (var), tipos (type) y funciones (func). Las expresiones se resuelven por precedencia (unario, multiplicativo, aditivo, relacional, && y ||) y los statements incluyen print/println, return, break, continue, asignaciones simples y compuestas, if (6 variantes), for (4 variantes) y switch (4 variantes). Por brevedad se remite al archivo MiniGo.g4 para el detalle completo de las reglas de producción.

1.3 Tipos de datos

Tipo	Descripción y ejemplo
int (y variantes)	Entero de 32 bits. Ej.: var x int = 42;
float64 / float32	Punto flotante. Ej.: var pi float64 = 3.14;
bool	Booleano; true/false son identificadores predeclarados.
rune	Carácter Unicode (alias de int32). Ej.: var c rune = 'A';
string	Cadena de texto (formatos raw e interpreted).
[N]T	Arreglo de tamaño fijo. Ej.: var a [5]int;
[]T	Slice (arreglo dinámico) con append, len y cap.
struct	Registro con campos. Ej.: type Point struct { x int; y int; };

1.4 Código que se puede y que no se puede escribir

Con Mini GO se pueden escribir variables globales y locales, structs, funciones con y sin retorno, slices con append, switch de múltiples casos, expresiones aritméticas, lógicas y a nivel de bit, control de flujo if/for/switch con break y continue, e inferencia de tipos con “:=” para tipos

primitivos. Las siguientes características de Go NO forman parte de Mini GO porque no están en la gramática:

Característica de Go	**En Mini GO**
Múltiples valores de retorno	No soportado
Goroutines y channels	No soportado
Interfaces y métodos sobre tipos	No soportado
defer / panic / recover	No soportado
Literales de struct y de map	No soportado
Punteros	No soportado
Funciones anónimas / closures e imports	No soportado
Inferencia de tipos no primitivos	Restringida a primitivos

1.5 Comparación con Go

Aspecto	**Go 1.22**	**Mini GO**
Punto y coma	Insertados automáticamente	Explícitos y obligatorios
true / false	Keywords	Identificadores predeclarados
Inferencia ":="	Cualquier tipo	Solo tipos primitivos
Slices	Memoria dinámica	{T*, len, cap} con malloc
Println	fmt.Println (librería)	Statement built-in
Múltiples return	Soportado	No soportado
Variable no usada	Error de compilación	Error de compilación

2. Soluciones e Implementación

2.1 Arquitectura general

El compilador sigue un pipeline por fases: código fuente → Lexer/Parser (ANTLR4) → AST → TypeChecker (visitor en C++) → Encoder (visitor que emite LLVM IR) → output.ll y output.o (objeto x86-64-64) → ejecución por JIT con lli o enlace a un ejecutable nativo. Está implementado

en C++17 con ANTLR4 4.13.2, la API C++ de LLVM y Qt6 para la interfaz gráfica. En la Figura 1 se resume este flujo.

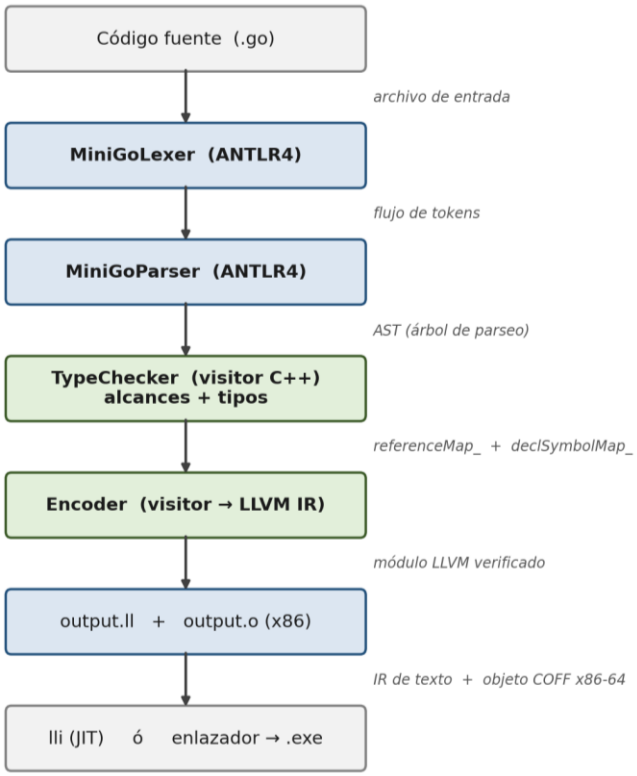


Figura 1: Pipeline de compilación por fases de miniGo

2.2 Scanner y Parser (ANTLR4)

A partir de MiniGo.g4, ANTLR4 genera el lexer, el parser y el visitor base [1]. El lexer reconoce 19 tokens de palabra reservada (14 keywords de control más 5 builtins: print, println, append, len y cap), todos los operadores multicácter (:=, <<=, &^=, etc.), literales enteros en cuatro bases, flotantes con exponente, runas, cadenas raw e interpreted [2], y comentarios de línea y de bloque. Un listener propio (MiniGoSyntaxErrorListener) usa las APIs de ANTLR4 (getExpectedTokens, getVocabulary) para producir mensajes precisos con sugerencias “did you mean?” y separa los errores léxicos de los sintácticos para evitar cascadas.

2.3 Tabla de símbolos

SymbolTable implementa scopes anidados mediante una pila de mapas. Los símbolos se guardan en un almacenamiento permanente para que los punteros Symbol* sigan siendo válidos después de cerrar un scope (requisito del Encoder). Se abren scopes para funciones, bloques, for, if-con-inicializador y switch-con-inicializador.

2.4 TypeChecker (chequeo de alcances y tipos)

Es un visitor que retorna información de tipo. En una primera pasada registra los tipos y las firmas de todas las funciones (lo que habilita llamadas hacia adelante y la recursión mutua, igual que en Go [2]) y luego visita los cuerpos. Verifica: redeclaración en el mismo scope, uso de identificadores no declarados, cantidad y tipo de argumentos, tipo de retorno, condiciones booleanas en if/for/switch, compatibilidad en asignaciones simples y compuestas, índices de tipo int, campos de struct, append/len/cap, inferencia solo para primitivos, ":=" con al menos una variable nueva, break/continue en contexto válido y variables locales declaradas pero no usadas. Todos los errores reportan fila y columna.

2.5 Encoder — generación de código LLVM IR y objeto x86-64

El Encoder genera LLVM IR con la API C++ [3]. Mapea int y rune a i32, float64 a double, bool a i1, string a puntero, [N]T a arreglo, []T a la estructura {T*, i32, i32} y struct a un tipo nombrado. Las globales se emiten como GlobalVariable y las locales y parámetros como allocas izadas al bloque entry; main() se emite como i32 con retorno 0. Implementa if/else, los cuatro for, switch como cadena de comparaciones, break/continue con pilas separadas, evaluación en corto circuito de && y || con PHINodes [4], aritmética entera y de punto flotante, operadores a nivel de bit y append con gestión real de capacidad (malloc + memcpy). El módulo se valida con verifyModule(): si la verificación falla, la compilación se reporta como fallida y no se emite código. Si el IR es válido, además del archivo output.ll se genera código de máquina real para x86-64 en output.o (objeto COFF x86-64-64) mediante TargetMachine, enlazable a un ejecutable nativo; el IR también puede ejecutarse de inmediato con lli. La Figura 2 muestra los bloques básicos generados para el control de flujo.

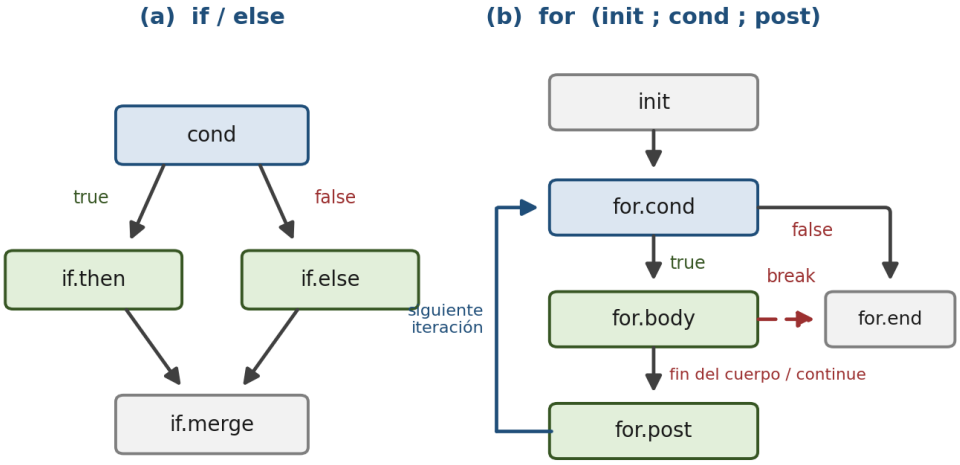


Figura 2: Bloques básicos LLVM generados para el control del flujo

Estos bloques se enlazan mediante saltos condicionales (CreateCondBr) e incondicionales (CreateBr); break y continue usan pilas separadas que apuntan al bloque de salida y al de incremento del lazo. En cuanto a las estructuras de datos dinámicas, como se muestra en la Figura 3, un slice se representa en memoria como {T*, len, cap} y append decide en ejecución si reutiliza el búfer o lo reasigna.

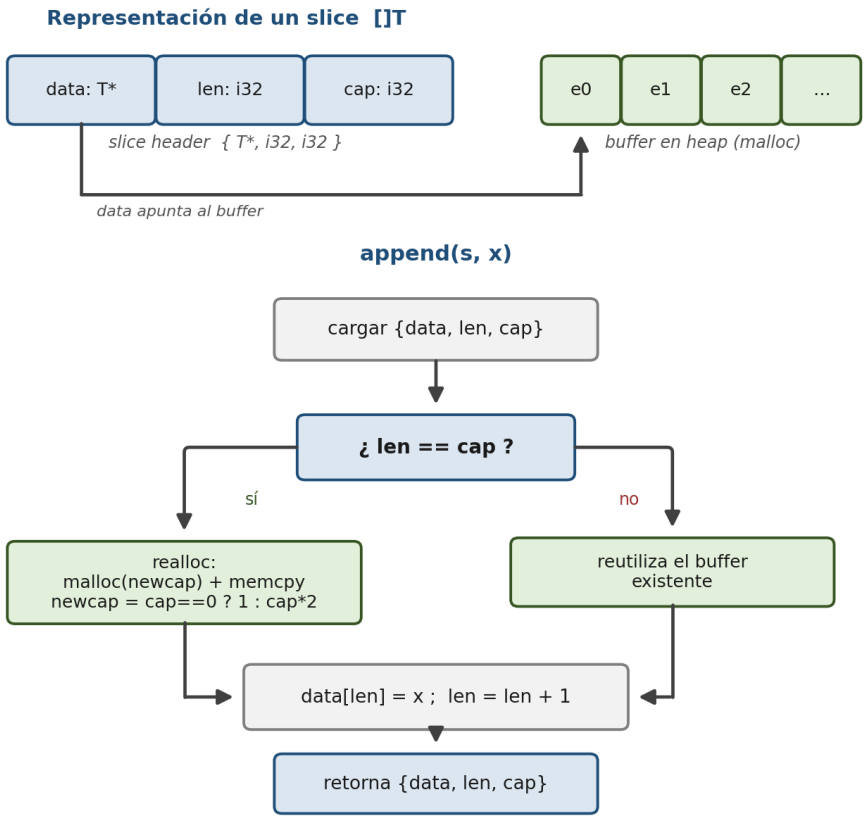


Figura 3: Estructura de un slice y lógica de append con realloc condicional.

2.6 Editor gráfico (Qt6)

El editor es un IDE mínimo con resaltado de sintaxis, numeración de líneas, resaltado de la línea actual, indicador de cambios sin guardar y posición del cursor [5]. Ofrece operaciones New/Open/Save/Save As con confirmación, Compile (Ctrl+Shift+B) y Compile & Run (Ctrl+B, que ejecuta con lli). El panel de errores usa colores por tipo (léxico, sintáctico, tipo y alcance), permite navegar al error con doble clic a la línea y columna exactas y con F2, e incluye búsqueda (Ctrl+F), ir a línea (Ctrl+G) y comentar/descomentar (Ctrl+/).

2.7 Manejo de errores

El sistema de errores es multicapa: léxicos (literales sin cerrar, caracteres inválidos), sintácticos (mensajes contextuales con el token esperado y sugerencias “¿did you mean?” por distancia de edición, con supresión de cascadas) y semánticos (tipos, alcances, retorno, break/continue y variable no usada). Todos los mensajes incluyen fila y columna.

3. Resultados Obtenidos

3.1 Componentes implementados

Componente	**Estado**
Scanner (tokens simples y complejos, comentarios, errores léxicos)	Completo
Parser (declaraciones, statements, expresiones, errores)	Completo
Tabla de símbolos (scopes anidados)	Completo
Patrón Visitor de ANTLR4	Completo
Chequeo de alcances (redeclaración, llamadas, argumentos, niveles, campos de struct, break/continue)	Completo
Chequeo de tipos (asignaciones, lógicas, argumentos, retorno, slices/arrays, structs, expresiones complejas)	Completo
Código generado (globales, funciones, locales/parámetros, expresiones de tipos simples incl. float, slices/arrays, if/switch, print/println/len/cap)	Completo
Código de máquina nativo x86-64 (output.o)	Completo
GUI (editor, compilación y manejo de archivos)	Completo

3.2 Funcionalidades no implementadas o limitadas

Funcionalidad	**Motivo e impacto**
Concatenación de strings en codegen	La gramática la permite sintácticamente (+ entre expresiones), pero MiniGo no contempla operar con cadenas (solo se imprimen con println), por lo que el TypeChecker la rechaza con un error de tipos indicando fila y columna.

Análisis de todos los caminos de retorno	Se verifica que exista al menos un return; el Encoder agrega un retorno implícito.
Selectores encadenados (p.a.b) en codegen	El TypeChecker los valida; el Encoder soporta un nivel de acceso.
Inicializadores globales no constantes	LLVM exige constantes; se inicializa a cero y se emite una advertencia.
Goroutines, interfaces, defer, punteros, closures, imports	Fuera de la gramática, por diseño del lenguaje.

4. Conclusiones

ANTLR4 simplificó el frontend al generar el lexer, el parser y el visitor de forma automática, lo que permitió concentrar el esfuerzo en el análisis semántico y la generación de código. Aun así, conseguir mensajes de error precisos con posición exacta y sugerencias útiles requirió trabajo adicional considerable por encima del comportamiento que ofrece el runtime por defecto.

El sistema de tipos de Mini GO, pese a ser un subconjunto de Go, bastó para ejercitar los conceptos clave del chequeo semántico: ámbitos anidados, inferencia para primitivos, compatibilidad entre tipos compuestos y verificación del retorno. Esto demostró que un lenguaje reducido puede ser suficientemente expresivo para validar una implementación semántica completa y robusta.

LLVM resultó un backend efectivo: su IR es verificable antes de emitir código, a partir de él se genera un objeto nativo x86-64-64 y lli permite la ejecución inmediata. Además, su modelo de representación facilitó implementar patrones avanzados como el cortocircuito de operadores lógicos con nodos PHI y el manejo correcto de la capacidad real de los slices.

El manejo de errores con fila, columna y sugerencias resultó tan valioso como el compilador mismo para el usuario, pues transforma una herramienta técnica en algo realmente utilizable. A su vez, la arquitectura por fases facilitó desarrollar, probar y refinar cada componente de forma independiente, reduciendo la complejidad del proyecto en su conjunto.

5. Bibliografía

[1] T. Parr y el Proyecto ANTLR, "ANTLR 4 — Documentación oficial," [En línea]. Disponible: https://github.com/antlr/antlr4/blob/master/doc/index.md

[2] The Go Authors, "The Go Programming Language Specification," Google LLC, [En línea]. Disponible: https://go.dev/ref/spec

[3] LLVM Project, "LLVM Language Reference Manual," [En línea]. Disponible: https://llvm.org/docs/LangRef.html

[4] LLVM Project, "My First Language Frontend (LLVM Tutorial)," [En línea]. Disponible: https://llvm.org/docs/tutorial/

[5] Qt Group, "Qt 6 Documentation," [En línea]. Disponible: https://doc.qt.io/qt-6/